



AdaGCRAG: Adaptive Graph-Chunk Retrieval for Lightweight RAG

Yanqiu Zhang^{1,2}, Zhen Xu^{1,2}(✉), DongDong Huo^{1,2}, Xiaokun Guo^{1,2},
Qihui Zhou¹, and Yan Zhang^{1,2}

¹ Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
{zhangyanqiu, xuzhen, huodongdong, guoxiaokun, zhouqihui, zhangyan}@iie.ac.cn

² School of Cyber Security, University of Chinese Academy of Sciences,
Beijing, China

Abstract. Retrieval-Augmented Generation (RAG) mitigates hallucinations of Large Language Models (LLMs) and improves factual accuracy by incorporating external knowledge during inference. However, existing RAG approaches face critical limitations in resource-constrained environments: (1) dense retrieval often introduces irrelevant or redundant content due to semantic overlap in vector space, and (2) graph-based methods still depend on sophisticated reasoning capabilities that exceed the capacity of Small Language Models (SLMs). To address these challenges, we propose **Adaptive Graph-Chunk RAG (AdaGCRAG)**, a lightweight, local, and query-adaptive RAG framework that integrates chunk-level retrieval with graph-structured reasoning. AdaGCRAG categorizes each input into *Simple Specific Question (Simple SQ)*, *Complex Specific Question (Complex SQ)*, or *Abstract Question (Abstract Q)* and adaptively controls the retrieval granularity and reasoning depth. It leverages Qwen3’s dual-mode reasoning to minimize computation for simple tasks while enabling graph-based multi-hop inference for complex ones. A triple-level reranking module filters noisy subgraphs before fusion with dense retrieval results. AdaGCRAG runs efficiently on a single RTX 3070 GPU, making it suitable for personal agents and offline assistants. Experiments across five Question Answering (QA) benchmarks show consistent improvements over state-of-the-art lightweight RAG systems, achieving improvements of 9.5%, 6.9%, and 6.8% on HotpotQA, MultiHopRAG, and MuSiQue, respectively, demonstrating its effectiveness under limited-resource settings. Our implementation is available at <https://anonymous.4open.science/r/AdaGCRAG>.

Keywords: Retrieval-Augmented Generation · Knowledge Graph · Large Language Model

1 Introduction

Large Language Models (LLMs) like GPT-4 [1], Llama 3.1 [7] and Qwen3 [23] have shown remarkable capabilities across a wide range of natural language processing tasks. However, their reliance on parametric memory inevitably leads to

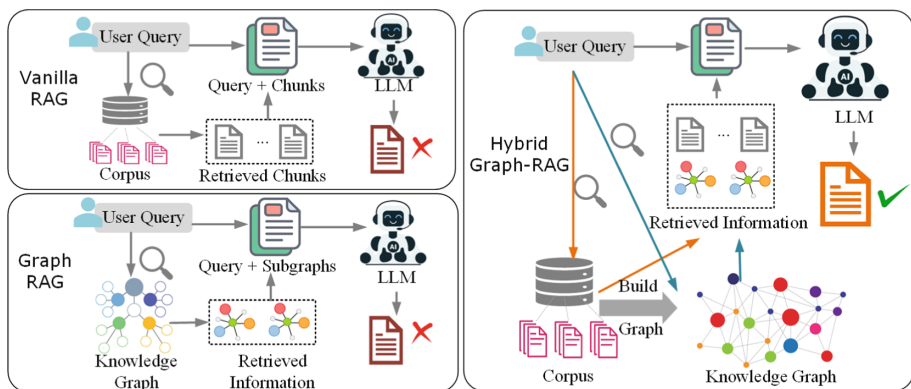


Fig. 1. The general workflow of various RAGs.

factual inaccuracies and hallucinations when dealing with knowledge-intensive questions [12]. Retrieval-Augmented Generation (RAG) has emerged as a powerful framework that enables LLMs to retrieve relevant external knowledge during inference, thereby enhancing answer faithfulness and up-to-date accuracy [4], as shown in Fig. 1. While conventional RAGs, which rely solely on dense vector similarity for document retrieval, like Vanilla RAG, often introduce irrelevant or redundant content and lack interpretability. As an improved solution, Graph-based RAGs, like LightRAG [9] and GraphRAG [5] incorporate structured Knowledge Graph (KG), representing information as graphs to capture relationships between entities. However, Graph-RAG architectures typically rely on LLMs with strong comprehension and reasoning abilities for effective query understanding, semantic alignment, and multi-hop inference, making them difficult to adapt to Small Language Models (SLMs) that lack such capabilities. This mismatch leads to sharp performance drops or complete system failure in lightweight settings [10, 29].

Towards this issue, MiniRAG [6] is a lightweight RAG designed for SLMs, utilizing semantic-aware heterogeneous graph indexing and topology-enhanced retrieval. However, its performance depends on accurate entity and relation extraction to construct the KG. Given that SLMs have limited capabilities in this area, MiniRAG’s real-world effectiveness may be compromised. In fact, not all queries require complex retrieval. Simple questions [27] can be answered with just a few documents, and using graph-based retrieval in such cases adds unnecessary noise and resource consumption. Therefore, hybrid approaches that selectively use chunk-based or graph-based retrieval based on query complexity can be considered an effective way to improve retrieval efficiency in SLM scenarios. However, existing hybrid methods, like HybridRAG [19], have successfully integrated knowledge from multiple sources, they still lack the capability to selectively retrieve or execute reasoning processes based on the type of query. While some systems use intent analysis models [8] to assess the query before executing RAG, this introduces additional model storage and computational overhead.

In short, existing RAGs suffer from: 1) Irrelevant or redundant information from retrieval; 2) Poor performance of graph-based RAG with SLMs; 3) Existing SLM-based RAG still heavily rely on high-quality KGs; 4) Hybrid methods lacking selective retrieval or reasoning based on query complexity, causing inefficiency. To address these limitations, we propose AdaGCRAG, a local, lightweight, and query-adaptive hybrid RAG framework that unifies dense chunk retrieval with graph-structured reasoning.

Unlike prior systems that treat all queries uniformly, AdaGCRAG categorizes each question into *Simple SQ*, *Complex SQ*, or *Abstract Q* using few-shot prompting. This classification guides both retrieval and reasoning. We adopt Qwen3 [23] models for their controllable reasoning modes: *non-thinking* for efficiency and *thinking* for complex tasks. For *Simple SQ*, AdaGCRAG performs chunk-level retrieval with reasoning disabled. *Complex SQ* adds graph-level evidence without reasoning. *Abstract Q* triggers both multi-hop graph expansion and Qwen3’s *thinking* mode. To reduce subgraph noise, triple-level reranking selects the most relevant triples before merging with chunks for final generation.

Our AdaGCRAG is optimized for local deployment and runs efficiently on consumer-grade hardware such as a single RTX 3070 GPU, making it highly suitable for personal agents and offline knowledge assistants. In addition to its modular and lightweight design, AdaGCRAG outperforms MiniRAG [6], a leading SLM-oriented RAG system, on all evaluated QA benchmarks. Specifically, it achieves a 6.9% accuracy gain on MultihopRAG [22], 9.5% on HotpotQA [27], and 6.8% on MuSiQue [24], demonstrating consistent improvements in both simple and complex reasoning tasks. These results highlight AdaGCRAG’s robustness and practical value in resource-constrained settings.

In summary, our main contributions are as follows:

- We introduce AdaGCRAG, a lightweight RAG framework that seamlessly combines chunk-based and graph-based retrieval, supporting efficient and accurate question answering in on-device or resource-limited settings.
- We propose a query-aware hybrid retrieval mechanism that adaptively modulates retrieval granularity and reasoning depth based on query complexity, balancing relevance with computational cost.
- We present a SLM-based RAG inference architecture that utilizes Qwen3’s dual-mode reasoning capabilities to intelligently balance the trade-off between speed and accuracy.
- We demonstrate superior performance across five diverse QA benchmarks under limited-resource conditions, showcasing AdaGCRAG’s robust generalization across various reasoning tasks and query complexities.

2 Related Work

LLM Prompting and RAG. Prompt engineering has become a key method to leverage the inherent abilities of LLMs without extensive fine-tuning [2, 16]. Prompt-based approaches allow LLMs to perform various tasks such as QA, summarization, and translation by simply conditioning the model with natural language prompts. However, LLMs’ parametric memory is inevitably limited by

outdated or incomplete knowledge, leading to hallucinated or unfaithful outputs [20]. To mitigate this, RAG integrates non-parametric memory through retrieval from external corpora [11, 13]. The standard RAG pipeline consists of two stages: (1) *retrieval*, where top- k relevant chunks are retrieved based on embedding similarity; and (2) *generation*, where the LLM generates answers conditioned on both the query and retrieved content. While effective in improving factual accuracy, these methods often suffer from shallow semantic matching [4].

Graph-Based RAG. To enhance entity-level precision and reasoning capabilities, recent research has integrated structured KGs into the RAG framework. ToG [21] treats the LLM as an agent that iteratively explores the KG via beam search to uncover reasoning paths. RoG [15] first generates relation paths grounded in the KG as faithful plans, which are then used to retrieve valid reasoning paths. SubgraphRAG [14] employs a lightweight multilayer perceptron with a parallel triple-scoring mechanism to retrieve subgraphs, encoding directional structural distances to enhance retrieval. AMAR [26] introduces a self-alignment module to align commonalities among entities, relations, and subgraphs, and a relevance gating module to determine the relevance of retrieved information. GNN-RAG [17] combines the reasoning abilities of Graph Neural Networks (GNNs) with the language understanding capabilities of LLMs. Despite improved factual consistency, these methods often depend on predefined schemas and high-quality KGs, limiting their adaptability in evolving environments. Additionally, many assume access to cloud-based LLMs, which poses challenges for privacy-preserving or local deployment.

Graph-Based RAG with Hybrid Retrieval. Graph-based RAG methods construct KGs from unstructured corpora to capture entity-centric relations and support multi-hop reasoning [28, 29]. These approaches generally follow a three-stage pipeline: (1) *KG Construction*, where an LLM extracts nodes and edges from corpus D to build graph G ; (2) *retrieval*, where relevant nodes/subgraphs are retrieved for query Q ; and (3) *generation*, where an LLM generates the answer based on Q and the retrieved context. GraphRAG [5] pioneered entity-centric KGs using community detection to enable multi-hop reasoning and global semantic retrieval. LightRAG [9] optimized efficiency through dual-level indexing and incremental graph updates, reducing computational costs by 40%. While effective, these methods typically require significant resources for graph construction, multi-hop retrieval, and large-scale LLM inference [29]. MiniRAG [6] targeted resource-constrained environments with heterogeneous graph indexing and topology-aware retrieval, achieving 75% storage reduction while maintaining accuracy. However, its graph construction and entity retrieval are imprecise, lacking adaptive strategies for different queries, which can affect the reliability.

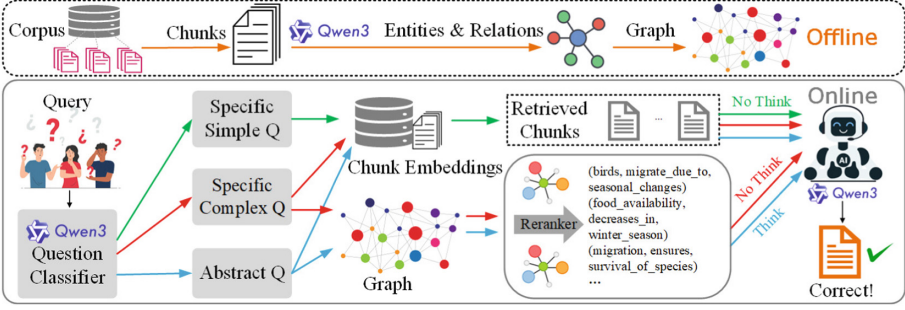


Fig. 2. AdaGCRAG consists of two phases: offline and online.

3 Preliminary

KGs store structured factual knowledge as triples $\mathcal{G} = \{(u, r, v) \mid u, v \in \mathcal{E}, r \in \mathcal{R}\}$, where $\mathcal{E}$ and $\mathcal{R}$ denote the sets of entities and relations, respectively.

Reasoning Subgraphs are task-relevant subgraphs extracted from a KG to support multi-hop reasoning. A reasoning subgraph is a connected set of triples $\mathcal{S}_q \subset \mathcal{G}$ constructed by traversing relation paths relevant to a given query q .

Example 1. A reasoning subgraph for the query “Where does Alice’s stepson work?” would be $\mathcal{S}_q = \{(Alice, marry\ to, Bob), (Bob, father\ of, Charlie), (Charlie, (work\ at, OpenAI))\}$

Hybrid Retrieval-Augmented QA extends traditional KGQA by incorporating both structured knowledge from KGs and unstructured evidence from chunks. Given a natural language question q , the system retrieves a reasoning subgraph $\mathcal{S}_q$ from $\mathcal{G}$ and a set of relevant text chunks $\mathcal{C}_q = \{c_1, c_2, \dots, c_p\}$ from a corpus of documents $\mathcal{D}$. The final model input is the hybrid context $\text{Input}(q) = (\mathcal{S}_q, \mathcal{C}_q)$, enabling joint reasoning over graph-structured and textual knowledge to produce the answer $a = f(q, \mathcal{S}_q, \mathcal{C}_q)$.

4 Approach

We present the detailed architecture of our proposed AdaGCRAG framework, as illustrated in Fig. 2, it consists of two phases: *offline* indexing and *online* retrieval. *Offline*, we use the LLM to extract entities and relations from raw texts to construct the KG, and create embedding indexes for both text chunks and KG entities and relations. *Online*, given a query, the LLM first performs question classification to guide adaptive retrieval. Retrieved information is then combined with the question and fed into the LLM to generate the final answer. The key steps of AdaGCRAG are summarized in Algorithm 1, which we will elaborate on in the following sections.

Algorithm 1: AdaGCRAg: Adaptive Graph-Chunk RAG Pipeline

Input: Corpus $\mathcal{D}$, Query q , Embedding model $\text{Embed}(\cdot)$, LLM
Output: Answer to query q

```

1 Offline Indexing: ;
2 foreach document  $D \in \mathcal{D}$  do
3   Segment into chunks  $C = \{c_i\}$ ;
4   Compute embeddings  $\mathbf{e}_i = \text{Embed}(c_i)$  ;
5   Extract entities/reasons via LLM; embed and store in  $\mathcal{V}_{\text{KG}}$  and graph DB ;
6 Online Retrieval: ;
7 Classify query  $q$  into type  $t \in \{\text{Simple}, \text{Complex}, \text{Abstract}\}$  via LLM ;
8 Retrieve top- $n$  text chunks using  $\mathbf{e}_q = \text{Embed}(q)$  ;
9 if  $t \neq \text{Simple}$  then
10   Extract query entities  $\mathcal{E}_q$ ; retrieve top- $k$  entities from  $\mathcal{V}_{\text{KG}}$  ;
11   Expand  $h$ -hop subgraph  $\mathcal{G}_q$ ; re-rank triples to get top- $m$  set  $\mathcal{T}_q$  ;
12 Evidence Combination: Combine  $\mathcal{T}_q$  and chunks based on  $t$  ;
13 Answer Generation: ;
14 if  $t = \text{Abstract}$  then
15   return  $\text{Qwen3}_{\text{think}}(q, \text{Evidence}(q))$  ;
16 else
17   return  $\text{Qwen3}_{\text{no-think}}(q, \text{Evidence}(q))$  ;

```

4.1 Offline Indexing

We process raw text documents to generate chunk embeddings and construct the KG. Let D represent a raw document, and $C = \{c_1, c_2, \dots, c_p\}$ the resulting set of text chunks. The segmentation begins with splitting the document by sentences, followed by further partitioning any long segments using a length threshold L . After obtaining the chunks, we generate embeddings for them and extract entities and relations from the raw text to construct the KG. This step enables the system to effectively combine both the unstructured text (chunks) and the structured knowledge (KG) for later retrieval.

Chunk Embedding Generation. Each chunk $c_i \in C$ is passed through a pre-trained text embedding model $\text{Embed}(c_i)$ to obtain a dense vector representation $\mathbf{e}_i \in \mathbb{R}^d$, where d is the dimensionality of the embedding space. The embeddings of all chunks are then stored in a vector database $\mathcal{V}_{\text{chunks}}$ for fast retrieval.

KG Construction. Given a chunk c_i , the LLM extracts entities $\mathcal{E}(c_i)$ and relations $\mathcal{R}(c_i)$ using a few-shot prompt P , adapted from prior work [5, 6]:

$$\mathcal{E}(c_i), \mathcal{R}(c_i) = \text{LLM}(c_i, P) \quad (1)$$

To ensure compatibility with lightweight models and reduce noise, we shorten the demonstration examples and refine the instructions. The prompt guides the model through three main steps: (1) identify all entities of specified types from the text; (2) extract all meaningful relations between entity pairs; and (3) return a unified list of extracted entities and relations in English. Each extracted entity $e_j \in \mathcal{E}(c_i)$ and relation $r_k \in \mathcal{R}(c_i)$ is then embedded using the same embedding model $\text{Embed}(\cdot)$:

$$\mathbf{e}_e = \text{Embed}(e_j), \quad \mathbf{e}_r = \text{Embed}(r_k) \quad (2)$$

The embeddings for entities and relations are stored in a separate vector database $\mathcal{V}_{\text{KG}}$. Additionally, the text-based entities and relations of the KG are stored in a graph database, such as Neo4j, to support efficient topological search and retrieval of reasoning subgraphs.

4.2 Online Retrieval

The online retrieval adaptively adjusts to the complexity and abstraction level of the user query, enabling efficient retrieval and accurate response generation. It consists of four modules: *LLM-based Question Classification*, *Hybrid Graph-Chunk Retrieval*, *Triple Re-ranking*, and *Graph-Augmented Reasoning with Hybrid Thinking*. This modular design allows our system to handle both factual and conceptual questions with a unified yet adaptive strategy.

LLM-Based Question Classification. To enable efficient and low-cost selection of retrieval and reasoning strategies for user queries, we first classify each query q into one of three predefined categories: *Simple SQ*, *Complex SQ*, and *Abstract Q*.

Prompt for Question Classification P_{cls}

—Goal—

Given a single question, classify it into one of the following categories:

- 0 – **Simple Specific:** The question asks for a specific fact or detail that can typically be answered using one or two text chunks. No reasoning across chunks is required.
- 1 – **Complex Specific:** The question asks for specific information but requires reasoning across multiple text chunks to find or synthesize the answer.
- 2 – **Abstract:** The question is conceptual in nature, possibly asking for summaries, high-level insights, themes, or general ideas. It does not focus on specific factual retrieval.

Respond only with one label: 0, 1, or 2./no_think

Examples:

- Question: What is Herman A. Barnett’s occupation?
Answer (0, 1, or 2): 0
-
- Question: What are the potential challenges of adopting biodiversity-friendly techniques in market farming?
Answer (0, 1, or 2): 2

Now classify this one:

Question: {question}

Answer (0, 1, or 2):

The LLM performs the classification using a structured few-shot prompt P_{cls} , and the predicted label guide the downstream retrieval and reasoning modules. We conducted an evaluation on a manually annotated set of 60 questions, where Qwen3-4B and GPT-4o achieved comparable near-perfect accuracy, both outperforming LLaMA3.1-8B. Considering efficiency and integration, we adopted the Qwen3 series models in our pipeline.

Hybrid Graph-Chunk Retrieval. We implement a dual-source hybrid retriever that combines the advantages of dense chunk retrieval with structured graph-based retrieval to cover both semantic similarity and knowledge-driven relevance.

Dense Chunk Retrieval. The user query q is encoded into a dense representation $\mathbf{e}_q \in \mathbb{R}^d$ using the $\text{Embed}(\cdot)$ function. We retrieve the top- u most relevant chunks $\{c_1, c_2, \dots, c_u\}$ by computing cosine similarity with stored embeddings in $\mathcal{V}_{\text{chunks}}$:

$$\text{Sim}(\mathbf{e}_q, \mathbf{e}_i) = \frac{\mathbf{e}_q \cdot \mathbf{e}_i}{\|\mathbf{e}_q\| \|\mathbf{e}_i\|}, \quad \mathbf{e}_i \in \mathcal{V}_{\text{chunks}} \quad (3)$$

Graph-Based Subgraph Retrieval. For more complex queries (*Complex SQ* and *Abstract Q*), we further extract a reasoning subgraph. We first use the LLM to extract entities from the query:

$$\mathcal{E}_q = \text{LLM}(q, P_{\text{ent}}) \quad (4)$$

P_{ent} instructs the model to identify all relevant entities of predefined types (e.g., organization, person, location, event) mentioned in the query. Each extracted entity $\text{ent}_i \in \mathcal{E}_q$ is embedded and matched against the entity vector database $\mathcal{V}_{\text{KG}}$. For the top- k matches, we apply a h -hop topological expansion on the graph database to retrieve the connected reasoning subgraph $\mathcal{G}_q$:

$$\mathcal{G}_q = \text{Expand}_h(\text{TopK-Match}(\mathcal{E}_q, \mathcal{V}_{\text{KG}})) \quad (5)$$

Triple Reranking over Subgraphs. To reduce noise in the topology-derived subgraph, we re-rank the triples in $\mathcal{G}_q = \{(u_j, r_j, v_j)\}$ based on their semantic relevance to the query, filtering out redundant information and enhancing reasoning precision. Each triple is linearized into a plain-text sequence using a simple concatenation scheme: " $\mathbf{u} \ \mathbf{r} \ \mathbf{v}$ ", where the head, relation, and tail are joined with spaces. For re-rank scoring, we utilize a compact cross-encoder model to directly estimate the semantic similarity between the query q and each concatenated triple string.

$$s_j = \text{Score}(q, (u_j, r_j, v_j)) = \text{CrossEncoder}(q, \text{str}(u_j, r_j, v_j)) \quad (6)$$

We select the top- m triples with the highest scores to construct $\mathcal{T}_q \subset \mathcal{G}_q$, a filtered set of the most semantically aligned knowledge statements for downstream reasoning.

Conditional Graph-Augmented Reasoning with Hybrid Thinking. To balance efficiency and reasoning capability, we leverage the hybrid inference architecture of Qwen3, which supports two distinct modes: *non-thinking* mode for rapid responses and *thinking* mode for step-by-step reasoning. This dual-mode design enables dynamic control of computational effort, governed by a query-specific reasoning budget. Before performing inference with the Qwen3 model, we conditionally integrate the unstructured and structured knowledge as follows:

$$\text{Evidence}(q) = \begin{cases} \text{Top-}n_s \text{ chunks,} & \text{if } \text{cls}(q) = \text{Simple SQ} \\ \text{Top-}k_c \text{ triples} + \text{Top-}n_c \text{ chunks,} & \text{if } \text{cls}(q) = \text{Complex SQ} \\ \text{Top-}k_a \text{ triples} + \text{Top-}n_a \text{ chunks,} & \text{if } \text{cls}(q) = \text{Abstract Q} \end{cases} \quad (7)$$

Here, k_* and n_* denote the number of triples and chunks retrieved for each query type, respectively. For reasoning-heavy tasks, triples are placed before chunks to emphasize graph-based reasoning. The final answer is generated as:

$$\text{Answer}(q) = \begin{cases} \text{Qwen3}_{\text{think}}(q, \text{Evidence}(q)), & \text{if } \text{cls}(q) = \text{Abstract Q} \\ \text{Qwen3}_{\text{no-think}}(q, \text{Evidence}(q)), & \text{otherwise} \end{cases} \quad (8)$$

We selectively activate Qwen3’s *think mode* based on the predicted query type. For *Specific* queries, the model uses *no-think* to provide fast answers without step-by-step reasoning. For *Abstract* queries, the model switches to *think* to enable multi-hop or conceptual reasoning over retrieved evidence.

5 Experiments

Our experiments aim to answer following research questions (RQs):

- *RQ1: Comparative Performance.* How does AdaGCRAg perform compared with existing baselines on specific and abstract QA tasks?
- *RQ2: Component Analysis.* What is the contribution of key components to AdaGCRAg’s overall effectiveness?
- *RQ3: Case Studies.* How effectively does AdaGCRAg respond to different types of complex queries?

5.1 Experiment Settings

Datasets. We evaluate AdaGCRAg on both *Specific* and *Abstract* QA tasks.

Specific QA involves fact-based questions grounded in explicit chunks or entities, and is divided into two types. *Simple* questions can be answered from one or two chunks without complex reasoning; we use HotpotQA [27]. *Complex* questions require multi-hop reasoning across documents and understanding of implicit relations; we use Multihop(RAG)QA [22] and MusiqueQA [24].

Table 1. Datasets used in our experiments.

Dataset	# Tokens	# Passages	# Questions	# Nodes	# Edges	QA Type
HotpotQA	1,284,956	9,221	1,000	43,809	48,714	Specific QA
MultihopRAG	1,426,396	609	2,556	45,310	51,831	Specific QA
MuSiQue	3,280,174	29,898	3,000	131,112	138,192	Specific QA
Mix	611,602	61	125	18,869	12,609	Abstract QA
MultihopSum	1,426,396	609	125	45,310	51,831	Abstract QA

Abstract QA requires high-level synthesis and conceptual understanding. We use Mix [18] and MultihopSum [22], which feature questions focused on broader themes, summaries, and topic-level reasoning.

The statistics of these datasets are reported in Table 1. As LLM-based graph construction is inherently non-deterministic, repeated inputs may yield variations in extracted triples. Thus, all reported node and edge statistics are derived from the KG generated by our AdaGCRAg(Qwen3-4B).

Baselines. We compare our AdaGCRAg against several representative model configurations.

- **Zero-Shot (Inference-only):** The LLM directly answers the question without any external retrieval.
- **Vanilla RAG (Chunks-only):** The top-ranked text chunks retrieved from the corpus are provided to the LLM for answer generation.
- **Graph-based RAG:** To ensure a fair and efficient comparison, we select graph-augmented methods that emphasize lightweight design and retrieval efficiency, including *LightRAG* [9], *HippoRAG* [10], and *MiniRAG* [6]. Among these, *LightRAG* supports local, global, and hybrid retrieval strategies (denoted as *LightRAG-L*, *LightRAG-G*, and *LightRAG-H*).

Evaluation Metrics. For Specific QA tasks, we evaluate model performance using *Accuracy* and *Recall*, where a prediction is considered correct if it contains the gold answer, without requiring exact string matches [3]. For Abstract QA, we adopt a head-to-head comparison approach following prior work [29], using GPT-4o as the evaluator. Specifically, we assess responses along four evaluation dimensions: *Comprehensiveness*, *Diversity*, *Empowerment*, and *Overall* for Abstract QA tasks.

Implementation Details. For text preprocessing, we first split documents by paragraph using newline characters (`\n`). If a paragraph exceeds a predefined chunk size L , it is further divided into smaller segments. We set the $L = 1200$ tokens with a 100-token overlap and store the resulting embeddings in the *nano*

Table 2. Comparison of methods on different datasets, where **Red** denotes the best result, and **Orange** denotes the best result excluding the best one.

Method	HotpotQA		MultihopRAG		MuSiQue	
	Acc.	Rec.	Acc.	Rec.	Acc.	Rec.
ZeroShot (Llama3.1-8B)	27.39	28.79	50.98	39.27	4.37	6.04
ZeroShot (Qwen3-4B w/o think)	22.68	22.74	53.99	38.44	2.90	7.54
VanillaRAG (Llama3.1-8B)	50.16	53.05	52.23	41.93	16.78	24.87
LightRAG-L (Llama3.1-8B)	33.47	38.16	45.14	38.01	8.37	19.85
LightRAG-G (Llama3.1-8B)	26.84	32.62	50.14	40.97	6.22	18.60
LightRAG-H (Llama3.1-8B)	34.94	40.31	52.42	43.09	10.03	20.36
HippoRAG (Llama3.1-8B)	50.65	54.13	54.76	45.35	17.42	26.04
LightRAG-H (Qwen3-4B w/o think)	36.18	38.76	53.90	39.17	9.70	22.49
MiniRAG (Qwen3-4B w/o think)	43.09	47.43	55.37	40.63	14.15	25.88
AdaGCRAg (Qwen3-4B)	52.61	54.97	62.28	60.89	20.94	29.26

vector base. For all retrieval-based methods requiring top- K selection (e.g., for chunks or entities), we default to $K = 4$ to stay within the context length limits. Our method supports query-adaptive retrieval, enabling more fine-grained control. Specifically, for *Simple SQ*, we perform chunk-based retrieval only and select the top-4 relevant chunks. For *Complex SQ*, we retrieve the top-4 chunks and the top-4 similar entities, expand to 2-hop neighbors in the KG, and select 25 candidate triples after reranking. For *Abstract Q*, we retrieve the top-6 chunks and the top-6 similar entities, apply a 2-hop graph expansion, and retain 30 triples after reranking for multi-hop reasoning. All hyperparameter choices, such as chunk size and expansion depth, are aligned with commonly adopted settings in prior studies [9, 29].

To maintain a lightweight system design, we use *all-MiniLM-L6-v2* [25] for embedding and *ms-marco-MiniLM-L6-v2* [25] for reranking. While larger models often improve retrieval quality, our focus is on evaluating the retrieval diversity of lightweight RAG systems rather than maximizing performance through heavier components. For generation, we use *Qwen3-4B* as the primary experimental model, and include *LLaMA3.1-8B*, *Qwen3-1.7B* and *Qwen3-0.6B* as additional comparison baselines. *Qwen3*-based experiments are conducted on a single RTX 3070 GPU to emulate local deployment scenarios, while *LLaMA3.1-8B*, due to its higher computational demands, is evaluated on an RTX 4090.

5.2 Evaluation for Specific QA

We report the metric values of various RAG strategies on Specific QA tasks in Table 2. We can make the following observations and analyses.

	GC0.6	GCL	GC1.7	MN4	GC4
GC0.6	50	5	1	2	0
GCL	95	50	32	29	17
GC1.7	99	68	50	32	3
MN4	98	71	68	50	24
GC4	100	83	97	76	50

(a) Comprehensiveness

	GC0.6	GCL	GC1.7	MN4	GC4
GC0.6	50	8	1	2	0
GCL	92	50	6	5	2
GC1.7	99	94	50	25	19
MN4	99	95	75	50	32
GC4	100	98	81	68	50

(b) Diversity

	GC0.6	GCL	GC1.7	MN4	GC4
GC0.6	50	10	2	2	1
GCL	90	50	17	15	10
GC1.7	98	83	50	39	26
MN4	98	85	61	50	38
GC4	99	90	74	62	50

(c) Empowerment

	GC0.6	GCL	GC1.7	MN4	GC4
GC0.6	50	5	1	0	0
GCL	95	50	18	12	8
GC1.7	99	82	50	28	10
MN4	100	88	72	50	28
GC4	100	91	90	72	50

(d) Overall

Fig. 3. The abstract QA results on MultihopSum dataset.

	GC0.6	GCL	GC1.7	MN4	GC4
GC0.6	50	6	4	3	0
GCL	94	50	27	22	19
GC1.7	96	73	50	30	6
MN4	97	78	70	50	31
GC4	100	81	94	69	50

(a) Comprehensiveness

	GC0.6	GCL	GC1.7	MN4	GC4
GC0.6	50	7	1	1	1
GCL	93	50	6	6	7
GC1.7	99	94	50	32	30
MN4	99	94	68	50	36
GC4	99	93	70	64	50

(b) Diversity

	GC0.6	GCL	GC1.7	MN4	GC4
GC0.6	50	11	0	2	0
GCL	89	50	19	14	11
GC1.7	100	81	50	35	23
MN4	98	86	65	50	35
GC4	100	89	77	65	50

(c) Empowerment

	GC0.6	GCL	GC1.7	MN4	GC4
GC0.6	50	7	0	1	0
GCL	93	50	18	13	10
GC1.7	100	82	50	31	16
MN4	99	87	69	50	32
GC4	100	90	84	68	50

(d) Overall

Fig. 4. The abstract QA results on Mix dataset.

(1) *Model Efficiency.* Qwen3-4B, with only half the parameters of LLaMA3.1-8B, achieves comparable performance in both ZeroShot and LightRAG-H settings. Its ability to run on a single 3070 GPU underscores the potential of lightweight LLMs for cost-sensitive and resource-constrained deployments.

(2) *Retrieval Design.* Using KG retrieval generally enhances the reasoning ability of LLMs. However, in frameworks such as LightRAG and MiniRAG, the KG functions primarily as an indexing mechanism: only passages linked to extracted entities are retained. In our experiments, we observe that both entity extraction and graph construction are inherently non-deterministic—regardless of whether LLaMA3.1-8B or Qwen3-4B is used. Consequently, the resulting KG structures vary across runs even with identical input, compromising retrieval consistency. Therefore, in lightweight RAG systems, preserving chunk-level retrieval is essential to ensure both coverage and stability.

(3) *Graph-Augmented Reasoning.* HippoRAG achieves strong performance across benchmarks by leveraging semantic retrievers such as Contriever or ColBERTv2 to identify paraphrastic entities and augment graph connectivity with synonym edges. It further applies Personalized PageRank (PPR) for evidence aggregation. While this pipeline improves reasoning capability, its complex graph construction and reranking mechanisms introduce considerable overhead, which may be unnecessary for simpler questions.

(4) *Adaptive Strategy.* For simple questions, introducing KGs may unnecessarily complicate the retrieval and suppress critical evidence, as evidenced by the inferior performance of LightRAG and MiniRAG on HotpotQA compared to Vanilla RAG. In contrast, our AdaGCRAg dynamically routes queries: it uses direct chunk retrieval for simple questions, and hybrid retrieval combin-

ing multi-hop KG reasoning with full-context chunk information for complex ones. This adaptive framework consistently outperforms baseline methods across multiple QA benchmarks.

Table 3. Ablation results of AdaGCRAG on MultihopRAG, showing accuracy and recall after removing reranking, triple-based, or chunk-based retrieval.

Model	AdaGCRAG		-Rerank		-Triples		-Chunks	
	Acc.	Rec.	Acc.	Rec.	Acc.	Rec.	Acc.	Rec.
Qwen3-4B	62.55	60.35	60.17	56.42	59.97	51.38	46.56	38.24
Qwen3-1.7B	54.56	53.32	51.03	49.17	51.43	45.19	40.12	32.65

5.3 Evaluation for Abstract QA

We evaluate the performance of various methods on abstract QA tasks as shown in Fig. 3 and Fig. 4, each cell indicates the proportion of questions for which the method in the row produces more accurate answers than the method in the column. Since MiniRAG is the only existing graph-based RAG specifically designed for deployment under resource-constrained environments, we use it as a strong baseline. For clarity, we denote *GC0.6*, *GCL*, *GC1.7*, *MN4*, and *GC4* as *AdaGCRAG (Qwen3-0.6B)*, *AdaGCRAG (LLaMA3.1-8B)*, *AdaGCRAG (Qwen3-1.7B)*, *MiniRAG (Qwen3-4B)*, and *AdaGCRAG (Qwen3-4B)*, respectively.

We highlight several observations: (1) When enabling the *thinking* mode in Qwen3, the reasoning ability improves significantly with model size, following the trend *Qwen3-4B* > *Qwen3-1.7B* > *LLaMA3.1-8B* >> *Qwen3-0.6B*. Combined with the ablation results in Sect. 5.5, we find that Qwen3-1.7B serves as a strong trade-off in severely resource-constrained environments. (2) Compared to MiniRAG, which retrieves chunks through entity-anchored graph traversal, AdaGCRAG performs dual-path retrieval by separately indexing and retrieving both graph-structured triples and raw text chunks. This allows AdaGCRAG to capture a broader range of relevant evidence and better support complex reasoning. When coupled with Qwen3’s internal reasoning capabilities, AdaGCRAG is able to more effectively integrate heterogeneous knowledge and generate more comprehensive responses. To the best of our knowledge, our method is the first to systematically explore abstract QA under constrained-resource settings in graph-based RAG. These results demonstrate the potential of scaling down both retrieval and generation while retaining robust multi-hop reasoning performance.

5.4 Detailed Analysis

To answer the question *RQ2* and assess the contribution of each core component in AdaGCRAG, we conduct the ablation study on MultihopRAG using two back-

bone LLMs: *Qwen3-4B* and *Qwen3-1.7B*. As shown in Table 3, we evaluate three major variants by selectively removing key modules from the full framework:

- *-Rerank*: Disables the graph-based reranking stage by directly truncating the top- m retrieved triples based on retrieved subgraphs.
- *-Triples*: Disables the KG retrieval, using only chunk-level retrieval.
- *-Chunks*: Disables chunk-level, using only the reranked triples from the KG.

The results yield several insights into AdaGCRAg’s design:

1. **Model Scalability.** Despite its smaller size, *Qwen3-1.7B* maintains strong retrieval and reasoning performance. This demonstrates its viability for deployment in resource-constrained local environments, offering a competitive trade-off between performance and efficiency.
2. **Component Contributions.** Both triple-based and chunk-based retrieval significantly impact overall performance. However, the absence of chunk information leads to a more severe degradation, highlighting the indispensable role of unstructured context even when structured KG evidence is available.
3. **Complementary Roles.** Triples and chunks serve complementary functions: triples enable precise multi-hop reasoning via topological query-to-entity path matching, while chunks provide detailed, unstructured descriptions that supplement the often sparse KG facts. Their synergy allows the LLM to balance structured reasoning with rich contextual grounding, enhancing the overall effectiveness.

5.5 Case Study Analysis

Before conducting the case study, we first analyzed the distribution of question types in each dataset. The statistics are summarized as follows:

- *HotpotQA*: 263 Simple SQ, 729 Complex SQ, and 8 Abstract Q.
- *MultihopRAG*: 17 Simple SQ, 2,448 Complex SQ, and 91 Abstract Q.
- *MuSiQue*: 146 Simple SQ, 2,823 Complex SQ, and 31 Abstract Q.

Based on dataset statistics, each dataset covers all three types of retrieval strategies, corresponding to the different question types. To illustrate AdaGCRAg practical advantages, we use two representative examples from Multihop, which targets both Specific and Abstract QA tasks, to perform case studies. As shown in results, it demonstrates AdaGCRAg selectively applies retrieval and structured reasoning depending on the question types.

Complex SQ. This case illustrates the limitations of Qwen3-4B: 1) Although the output was constrained to be brief, Qwen-4B (ZeroShot-nothink) frequently ignores the instruction, producing verbose and often inaccurate content. By contrast, AdaGCRAg benefits from targeted prompts and adaptive retrieval to provide accurate, faithful responses. 2) Even with step-by-step reasoning, Qwen-4B (ZeroShot-think) fails to identify the correct entity without external knowledge. However, AdaGCRAg retrieves relevant facts and integrates them into its reasoning to accurately identify the correct answer.

Abstract Q. Although Qwen3-4B shows stronger performance than LLaMA 3.1-8B on abstract questions in the *thinking* mode, it often relies on vague or speculative prior knowledge, producing responses that are fluent but not grounded. In contrast, AdaGCRAg makes use of retrieved evidence during its reasoning process, producing answers that are both coherent and factually reliable.

Case study for Complex SQ and Abstract Q

• **Complex SQ:**

Who is the individual that not only convinced a former Jane Street employee to join his trading and development ventures but also planned for a specialized board for his company, while facing allegations of fraud and conspiracy in court, as reported by Fortune, CNBC, and TechCrunch?

• **Ground-Truth:**Sam Bankman-Fried

• **ZeroShot w/o think Response:**

The individual in question is **Peter Thiel**. He convinced a former...

• **ZeroShot think Response:**

Rajon R. Suri

• **AdaGCRAg Response:**

Sam Bankman-Fried

• **Abstract Q:**

How does the dataset cover rumors about OpenAI's potential chip production?

• **ZeroShot think Response:**

<think>
Okay, the user is asking how the dataset covers rumors about OpenAI's potential chip production. First, I need to recall... including the GPT series, but **I don't remember them having a significant focus on chip production...**

</think>
The dataset covering rumors about OpenAI's potential chip production **would likely be** structured to track...

• **AdaGCRAg Response:**

<think>
Okay, the user is asking how the dataset covers rumors about OpenAI's potential chip production. Let me start by **recalling the information provided...** mention the rumors about OpenAI developing their own chips...

</think>
The dataset addresses rumors about OpenAI's potential chip production through... the dataset frames the rumors as part of a larger narrative about OpenAI's strategic positioning in AI...

Summary. The case study analysis showcases AdaGCRAg's ability to adapt its retrieval strategies to a wide variety of question types. Without modifying system-level logic, changes in the input question alone are sufficient to trigger distinct and appropriate RAG pipelines, underlining the flexibility and robustness of our hybrid retrieval-augmented framework.

6 Conclusion

We present AdaGCRAg, a lightweight yet effective retrieval-augmented generation framework that combines dense chunk retrieval and structured KG reasoning in a query-adaptive manner. By classifying questions into simple, complex, and abstract categories, AdaGCRAg dynamically selects appropriate retrieval and

reasoning strategies to balance efficiency and accuracy. Extensive experiments across both specific and abstract QA tasks demonstrate that AdaGCRAg significantly outperforms existing lightweight baselines, particularly under resource-constrained settings. Through a modular pipeline and conditional inference using Qwen3, our method achieves flexible, interpretable, and high-performance RAG without requiring cloud-based LLMs or costly graph construction.

Supplemental Material Statement. Our datasets, source code, and other details are available online at <https://anonymous.4open.science/r/AdaGCRAg>.

References

1. Achiam, J., et al.: Gpt-4 technical report. arXiv preprint [arXiv:2303.08774](https://arxiv.org/abs/2303.08774) (2023)
2. Agarwal, R., et al.: Many-shot in-context learning. *Adv. Neural. Inf. Process. Syst.* **37**, 76930–76966 (2024)
3. Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-rag: learning to retrieve, generate, and critique through self-reflection. In: *The Twelfth International Conference on Learning Representations* (2023)
4. Cheng, M., et al.: A survey on knowledge-oriented retrieval-augmented generation. arXiv preprint [arXiv:2503.10677](https://arxiv.org/abs/2503.10677) (2025)
5. Edge, D., et al.: From local to global: a graph rag approach to query-focused summarization. arXiv preprint [arXiv:2404.16130](https://arxiv.org/abs/2404.16130) (2024)
6. Fan, T., Wang, J., Ren, X., Huang, C.: Minirag: towards extremely simple retrieval-augmented generation. arXiv preprint [arXiv:2501.06713](https://arxiv.org/abs/2501.06713) (2025)
7. Grattafiori, A., et al.: The llama 3 herd of models. arXiv preprint [arXiv:2407.21783](https://arxiv.org/abs/2407.21783) (2024)
8. Guo, Q., Xie, X., Liu, S., Hu, M., Li, X., Bu, L.: Intention is all you need: refining your code from your intention. arXiv preprint [arXiv:2502.08172](https://arxiv.org/abs/2502.08172) (2025)
9. Guo, Z., Xia, L., Yu, Y., Ao, T., Huang, C.: Lightrag: simple and fast retrieval-augmented generation. arXiv preprint [arXiv:2410.05779](https://arxiv.org/abs/2410.05779) (2024)
10. Gutiérrez, B.J., Shu, Y., Gu, Y., Yasunaga, M., Su, Y.: Hipporag: neurobiologically inspired long-term memory for large language models. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems* (2024)
11. Izacard, G., Grave, E.: Leveraging passage retrieval with generative models for open domain question answering. In: Merlo, P., Tiedemann, J., Tsarfaty, R. (eds.) *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pp. 874–880. Association for Computational Linguistics, Online (2021). <https://doi.org/10.18653/v1/2021.eacl-main.74>. <https://aclanthology.org/2021.eacl-main.74/>
12. Ji, Z., et al.: Survey of hallucination in natural language generation. *ACM Comput. Surv.* **55**(12), 1–38 (2023)
13. Lewis, P., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. *Adv. Neural. Inf. Process. Syst.* **33**, 9459–9474 (2020)
14. Li, M., Miao, S., Li, P.: Simple is effective: the roles of graphs and large language models in knowledge-graph-based retrieval-augmented generation. In: *International Conference on Learning Representations* (2025)
15. Luo, L., Li, Y.F., Haffari, G., Pan, S.: Reasoning on graphs: faithful and interpretable large language model reasoning. In: *The Twelfth International Conference on Learning Representations* (2024)

16. Luo, M., Xu, X., Liu, Y., Pasupat, P., Kazemi, M.: In-context learning with retrieved demonstrations for language models: a survey. arXiv preprint [arXiv:2401.11624](https://arxiv.org/abs/2401.11624) (2024)
17. Mavromatis, C., Karypis, G.: GNN-RAG: graph neural retrieval for large language model reasoning. arXiv preprint [arXiv:2405.20139](https://arxiv.org/abs/2405.20139) (2024)
18. Qian, H., Zhang, P., Liu, Z., Mao, K., Dou, Z.: Memorag: moving towards next-gen rag via memory-inspired knowledge discovery, **1** (2024). arXiv preprint [arXiv:2409.05591](https://arxiv.org/abs/2409.05591)
19. Sarmah, B., Mehta, D., Hall, B., Rao, R., Patel, S., Pasquali, S.: Hybridrag: integrating knowledge graphs and vector retrieval augmented generation for efficient information extraction. In: Proceedings of the 5th ACM International Conference on AI in Finance, pp. 608–616 (2024)
20. Shi, W., Han, X., Lewis, M., Tsvetkov, Y., Zettlemoyer, L., Yih, W.t.: Trusting your evidence: Hallucinate less with context-aware decoding. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, vol. 2: Short Papers, pp. 783–791 (2024)
21. Sun, J., et al.: Think-on-graph: deep and responsible reasoning of large language model on knowledge graph. In: The Twelfth International Conference on Learning Representations (2024)
22. Tang, Y., Yang, Y.: Multihop-rag: benchmarking retrieval-augmented generation for multi-hop queries. arXiv preprint [arXiv:2401.15391](https://arxiv.org/abs/2401.15391) (2024)
23. Team, Q.: Qwen3: Think deeper, act faster (2025). <https://qwenlm.github.io/blog/qwen3/>. Accessed 02 May 2025
24. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: musique: Multihop questions via single-hop question composition. *Trans. Assoc. Comput. Linguist.* **10**, 539–554 (2022)
25. Wang, W., Wei, F., Dong, L., Bao, H., Yang, N., Zhou, M.: Minilm: deep self-attention distillation for task-agnostic compression of pre-trained transformers. *Adv. Neural. Inf. Process. Syst.* **33**, 5776–5788 (2020)
26. Xu, D., et al.: Harnessing large language models for knowledge graph question answering via adaptive multi-aspect retrieval-augmentation. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 39, pp. 25570–25578 (2025)
27. Yang, Z., et al.: Hotpotqa: a dataset for diverse, explainable multi-hop question answering. arXiv preprint [arXiv:1809.09600](https://arxiv.org/abs/1809.09600) (2018)
28. Zhang, Q., et al.: A survey of graph retrieval-augmented generation for customized large language models. arXiv preprint [arXiv:2501.13958](https://arxiv.org/abs/2501.13958) (2025)
29. Zhou, Y., et al.: In-depth analysis of graph-based rag in a unified framework. arXiv preprint [arXiv:2503.04338](https://arxiv.org/abs/2503.04338) (2025)